PRODUCT MANUAL + ENGINEERING HANDBOOK

AssignmentTracker

Every feature.

How it works.

How to explain it.

Prepared for Sarveshwar Senthil Kumar

Source-based edition | October 26, 2026

React / Express / MongoDB / PDF tools / Gemini

A FIELD GUIDE FOR USING AND PRESENTING THE PROJECT

Feature-by-feature instructions, architecture and data flow, demonstration sequence, interview questions, design tradeoffs, limitations, source references and implementation inventories.

Implementation descriptions are not runtime-verification claims.

CONTENTS

Find your workflow

Linked entries and PDF bookmarks let you jump to a chapter. The final appendices are extracted from the current source registry and route declarations.

How to use this handbook . . . 4
Your interview pitch . . . 5
Architecture: the whole system . . . 6
Data model and persistence . . . 7
Setup, configuration and deployment boundaries . . . 8
Home, navigation and global search . . . 9
Tasks: everyday planning . . . 10
Task relationships and attached evidence . . . 11
Folders, descriptions and folder media . . . 12
Calendar: a unified view of several sources . . . 13
Calendar: recurrence, colors, categories and undo . . . 14
Public TXST events and the scraper . . . 15
Sports and F1 season import . . . 16
Canvas synchronization and ICS import . . . 17
Document library and supported formats . . . 18
Typed notes and the writing studio . . . 19
Document creation: blank, Markdown and AI . . . 20
PDF editor: annotation fundamentals . . . 21
PDF text: editing, fonts and copying . . . 22
PDF pages, views and productivity tools . . . 23
Pen, touch, mouse and rendering performance . . . 24
Mathematics, calculator, function scanning and graphs . . . 25
OneNote imports and preserving handwriting . . . 26
AI assistant: conversations and user experience . . . 27
AI context, credentials, access and diagnostics . . . 28
AI document editing and PDF cleanup . . . 29
Jobs: links, screenshots, PDFs and local extraction . . . 30
Cover letters: reference voice and template fidelity . . . 31
Resumes and company-specific tailoring . . . 32
Custom workspace and task visualization . . . 33
Study stars and learning workflows . . . 34
Account, authentication and security boundaries . . . 35
Downloads, recovery and the meaning of Save . . . 36
Separate daily-events site and QR poster . . . 37

A practical eight-minute demonstration 38
Interview questions: architecture and data 39
Interview questions: PDF, input and performance 40
Interview questions: AI, extraction and trust 41
Preparing your own engineering stories 42
Limitations, testing plan and roadmap 43
Troubleshooting and demonstration recovery 44
Source map and feature inventory 45
Keyboard shortcut reference 46
API implementation inventory 50
Rapid interview cheat sheet 57

CHAPTER 01

How to use this handbook

This is a source-based guide to Sarveshwar Senthil Kumar's AssignmentTracker, prepared on October 26, 2026. It combines a user manual, an architecture explanation, a demonstration plan and interview preparation. It describes implementation, not a claim that every feature has passed runtime testing or works with every external service.

Three ways to read it

- Before an interview: read the pitch, architecture, demo, interview questions and limitations chapters first.
- While demonstrating: follow the demo sequence and use the feature chapters to explain individual controls.
- When answering technical follow-ups: use the source map and API appendix to find the implementation behind a claim.

Important distinctions

- Saved in the platform is different from downloaded to a local file. PDF annotations can be stored separately until export.
- Account preferences and records live in MongoDB; some display preferences and recovery drafts live only in the browser.
- Local parsing and calculation are different from AI recognition. The app uses both, with explicit boundaries.
- An undo facility is scoped to a workflow. There is no universal rollback of every application operation.
- The presence of test files is not evidence they pass. No application tests, builds, live API probes or browser QA were run to prepare this guide, following the repository's workflow preference.

How to speak credibly

Use the sample answers as a starting point, not a script to memorize. Say "the implementation uses..." for code-backed behavior. Use "I measured..." or "users reported..." only when you have that evidence. Describe personal authorship and collaboration accurately. Avoid invented user counts, performance improvements, deployment history or business outcomes.

Code takes precedence over old documentation. Some earlier README sections describe previous versions: the assistant can now edit supported open documents, notes have a Markdown studio, supported DOCX editing preserves the original package, and Ctrl/Cmd+K opens global search.

Sources: README.md; frontend/src/App.jsx; frontend/src/components/DocumentEditor.jsx; backend/assistantEditor.js.

CHAPTER 02

Your interview pitch

A 30-second answer

"AssignmentTracker is a full-stack student workspace that connects tasks, calendars, documents and career preparation. It started around organizing academic work and expanded into an annotated PDF editor, typed notes, campus-event discovery and AI-assisted document workflows. The frontend uses React, the API uses Express, and MongoDB stores account-owned records. A key design principle is keeping related work connected while making destructive and AI-powered actions explicit."

A 90-second answer

"The problem is that students move between assignment lists, calendars, notes, files and job applications. This app brings those workflows together. Tasks can have deadlines, dependencies and attachments; the calendar combines deadlines with personal entries, Canvas data and public events. The document library supports typed notes and an annotation-heavy PDF editor. Career tools import job details and tailor letters or resumes from existing documents.

"Technically, it is a React/Vite client talking to an Express/Mongoose backend. Files use the existing document collection, and relationships connect them to tasks, events and folders. Some operations, such as deletion recovery and calendar undo, use MongoDB transactions. AI is deliberately bounded: local code handles parsing and calculation where possible, while Gemini supplies language generation or recognition with account-level access controls.

"The most interesting engineering problems are preserving annotations across page changes, distinguishing pen input from navigation, maintaining document formatting, and preventing stale async results from overwriting newer edits. I would demonstrate those workflows, explain their limitations, and discuss the next steps in testing, storage and maintainability."

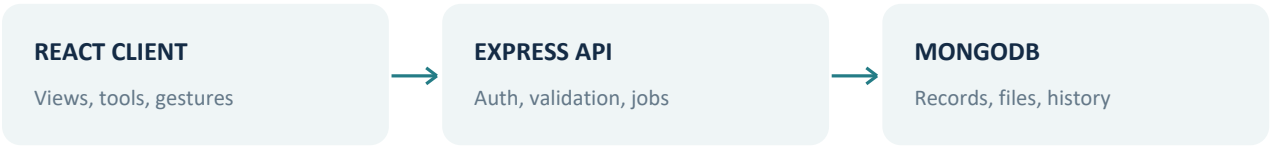
What makes the project distinctive

- Connected academic and career workflows, rather than unrelated standalone tools.
- Both local-first document utilities and optional AI assistance.
- Recovery designed into several destructive workflows.
- Difficult browser interactions: PDF geometry, text extraction, handwriting input and long conversion jobs.

Do not call it a real-time collaborative editor, autonomous agent, perfect PDF word processor or universally accurate job scraper. Those descriptions exceed the implementation.

CHAPTER 03

Architecture: the whole system



LOCAL UTILITIES

PDFs, Markdown, OCR, calculation, graphs

EXTERNAL BOUNDARIES

Gemini, Canvas, public events, OAuth

The main request path

Browser action -> React state -> authenticated /api request -> Express validation -> Mongoose owner-scoped query -> MongoDB -> response -> React refresh or optimistic update.

The Vite development server runs on port 3000 and proxies /api to the backend on port 5000. Hash routes such as #/tasks and #/documents let reloads and Back/Forward navigation work without a server route for every page. App.jsx coordinates shared tasks, folders, documents and the mounted editor. Feature-specific components and backend modules implement the individual workflows.

Where computation happens

- Browser: document viewing, annotation gestures, calculator/graph evaluation, Markdown rendering and PDF conversion, local job-file parsing/OCR, and interface state.
- Server: authentication, database writes, ownership checks, transaction history, remote website retrieval, AI requests, encryption of saved AI credentials and OneNote integration.
- Database: user accounts, tasks, folders, files, calendar records, chats, study items, jobs, generated drafts and recovery/index records.
- External services: Gemini, Canvas, public TXST event discovery, F1 schedule data and optional account-linking providers.

Three useful distinctions

UI state is not necessarily persistent data. A selected page or open menu is temporary; a task update is persisted. Browser preferences are not automatically account-wide settings. Public source caches are not private account records.

This is a modular application on one backend, not a fleet of microservices. Some coordination is per process, so multiple backend instances would require shared locking, rate limits and caches. MongoDB transactions require Atlas or a replica set.

Interview explanation: "I keep interactive rendering close to the browser and credentials and persistence on the server. I use structured records for relationships, and transaction snapshots where a destructive mutation must be recoverable. The tradeoff is coordination complexity as the feature set grows."

Sources: frontend/vite.config.js; frontend/src/App.jsx; backend/server.js; backend/calendarHistory.js.

CHAPTER 04

Data model and persistence

Core records

- User: identity, password hash, preferences, AI access, encrypted Gemini credential, calendar settings, workspace layout and connected-account metadata.
- Todo: title/text, status, description, deadline, folder, ordering, external source identifiers, links and prerequisite IDs.
- Folder and PDFFolder: separate task/document hierarchies, name, color, optional description and attachment IDs.
- PDF: the legacy collection name for all document formats; stores names, MIME type, binary data, annotations, tags, folder membership, task relationships, original backup and Trash state.
- CalendarEntry: personal appointments, date markers and sports events, linked task/document IDs, category, dates and optional recurrence grouping.
- AssistantConversation: messages, source links, file references, settings, title, archive/pin state and generation revisions.
- SavedJob and StudyItem: job evidence and structured starred learning material.

Supporting records

TaskDeletionRecovery preserves deleted task groups with expiration. CalendarChange records before/after changes. Generated resume and letter drafts are temporary records. DocumentSearchIndex stores extracted searchable content. OAuth state records are short-lived and single-use.

Why references matter

A document can be linked from several tasks or folder descriptions without uploading another binary copy. Unlinking does not delete the file. Stable task IDs make restored task references usable again. When a folder is deleted, its contents and child folders are promoted rather than destroyed.

Storage tradeoff

Keeping binary files in MongoDB makes ownership and backup behavior straightforward at this size, but large documents, backups and annotation arrays can approach document and memory limits. A production evolution would store binaries in object storage, retain metadata in MongoDB and stream uploads/downloads. That is a proposed improvement, not the current architecture.

Sources: backend/server.js; backend/folderAttachments.js; backend/taskDeletionRecovery.js; backend/assistantConversations.js; backend/globalSearch.js.

CHAPTER 05

Setup, configuration and deployment boundaries

Run the main application

From the repository root, install dependencies in backend and frontend with npm install. Copy backend/.env.example to backend/.env and configure MONGODB_URI and a strong JWT_SECRET. Start backend with npm run dev in one terminal and frontend with npm run dev in another. Open http://localhost:3000. Use Node.js 18 or newer as required by the existing backend code; choose a supported runtime for your own deployment.

Features with extra setup

- Transactions: use MongoDB Atlas or a replica set for undo, reordering, folder operations, attachments and other transaction-backed writes.
- AI: save a Gemini key in Account, or configure GEMINI_API_KEY on the backend. GEMINI_MODEL selects the provider model; compatible model availability and quotas must be verified in the provider account.
- Credential encryption: configure a stable AI_KEY_ENCRYPTION_KEY, or understand the fallback derived from JWT_SECRET. Changing these secrets can invalidate stored encrypted keys.
- OAuth: configure GitHub/LinkedIn client credentials, exact callback URLs and public application/API URLs. Production account linking expects HTTPS with a same-host /api proxy.
- OneNote: native preservation needs desktop OneNote on the Windows backend machine; compatibility conversion has its own Python requirements.

Before a live demonstration

Prepare a working account, a small PDF, a typed note, sample tasks and a saved job. Confirm optional integrations yourself. Keep a local-only demonstration path if the network or AI provider is unavailable. Never display .env files, tokens, private documents or a real key on a projector.

Deployment instructions in the repository describe hosting options; they do not establish that a public deployment is currently running. No installation, backend start or live integration check was performed for this handbook.

Sources: backend/.env.example; frontend/vite.config.js; backend/package.json; frontend/package.json; SETUP.md.

CHAPTER 06

Home, navigation and global search

Home and navigation

Home is the signed-in landing page. It displays open-task/deadline counts, a focus list, completion progress, a seven-day agenda, recent documents and career shortcuts. Actions open the underlying task, file or calendar item. Empty and loading states reflect available records instead of invented activity.

The compact navigation keeps Home, Workspace, Tasks, Calendar and Documents prominent. Career groups Resumes, Cover letters and Saved jobs. Tools contains the assistant, Study stars, Visualization, Canvas and keyboard shortcuts. The profile menu exposes Account and sign out. Smaller layouts use expandable navigation; dropdowns support keyboard focus and dismissal.

Exhaustive search, with explicit coverage

Ctrl/Cmd+K opens global search, including from editors and fullscreen PDF mode. It searches page features, shortcuts, tasks, document names/tags/content, personal events, jobs, chat messages, study material, folder paths/descriptions and calendar categories. Public campus events are included through their existing loading hook. Trash is optional. Results route to the relevant editor, task, event, conversation or folder.

Document content indexing runs in bounded worker threads. The index version tracks saved file updates; outdated index text should not be treated as current. Reindex rebuilds the account's extracted text. Pagination prevents returning every result in one response. Search includes supported PDF text/annotations, DOCX, workbook and text content; it does not magically OCR every scanned image or index unsaved edits, audio speech or all embedded media.

Interview explanation: "Search crosses different entity types, so I preserve each result's type and ID and route to the correct destination. Text extraction is isolated from request handling with bounded workers. The main tradeoff is freshness: saved document content must be indexed, and unsupported content has explicit coverage limits."

Sources: AppNavigation.jsx; HomePage.jsx; GlobalSearch.jsx; backend/globalSearch.js; backend/searchTextWorker.js.

CHAPTER 07

Tasks: everyday planning

What the user can do

- Create tasks in Inbox or a chosen folder, edit titles and descriptions, and set deadlines and supporting links.
- Advance between To Do, In Progress and Completed; reopen completed work.
- View all tasks or a folder, search/filter the visible work, and choose sorting such as deadline, status or custom order.
- Expand cards or open task details for richer editing, connections and attachments.
- Use compact task titles by default and toggle full wrapping when needed.
- Select multiple tasks for bulk deletion, or delete an individual task with the trash control.
- Disable the separate In Progress column in Account -> Task display. Those tasks remain alongside To Do tasks and retain their status.

Dragging and ordering

Custom order supports drag and drop. A compact floating preview and insertion line communicate placement. Dropping across columns can change status. Pointer visuals are updated with requestAnimationFrame rather than causing a full React render for every pointer move. Wheel/trackpad scrolling remains available, and edge scrolling helps reach distant destinations. Saving errors roll back optimistic ordering; Escape cancels a drag.

Deletion recovery

Successful task deletions create account-owned recovery snapshots for seven days. Undo deletion appears below the page heading, works after reload and can be repeated for earlier groups. It covers individual deletions, bulk selections and clear actions. Restoration preserves IDs, details, dates, order and dependency links; a missing original folder sends the task to Inbox. Account reset intentionally clears these snapshots.

Interview follow-up: "The status column is a view, not the identity of a task. Hiding a column should not silently delete or lose access to its tasks. For deletion I snapshot the mutation in a database transaction, which is different from a temporary client-side toast."

Sources: frontend/src/App.jsx; useTaskPointerDrag.js; TaskDeletionUndo.jsx; TaskDisplaySettings.jsx; backend/taskDeletionRecovery.js.

CHAPTER 08

Task relationships and attached evidence

Prerequisites and follow-ups

Task connections let one task come before or after another, including across folders. Links save immediately and are visible from either endpoint. They are advisory: the app does not block starting or completing a task just because prerequisites remain open. Duplicate/self-links are rejected or deduplicated, and circular chains are flagged for review.

The visualization turns prerequisites into arrows. Sequence layout arranges acyclic chains; cycles and unresolved sequences are handled separately. Deleting a task removes its dependency references, and recovery can restore appropriate connections.

Files and media

Task details and expanded editors have Files & media. Upload multiple files, drop files onto the panel, choose a document destination folder, or attach an existing library document. Images, audio and video support authenticated previews. Other formats open through the document editor or can be downloaded. Each upload is limited to 10 MB.

An existing document can be referenced by several tasks. Insert in description adds a stable document reference rather than copying the binary or freezing its filename. Library rows and the task map expose related tasks/documents. Unlinking an attachment keeps the library file, and Trash makes a reference unavailable until the document is restored.

How to explain the design

"I model relationships with IDs so a renamed document still resolves correctly. A relationship update and a document upload are different actions; deleting a relationship should not destroy shared data. For prerequisites, I chose advisory links rather than enforcing a scheduling engine, which makes the workflow flexible while still exposing cycles."

This is not automatic project scheduling or critical-path optimization. Do not claim that the graph guarantees a feasible plan, detects every business dependency, or assigns resources.

Sources: TaskDependencies.jsx; TaskAttachments.jsx; frontend/src/lib/taskDependencies.js; backend/taskDependencies.js; backend/taskDeletionRecovery.js.

CHAPTER 09

Folders, descriptions and folder media

Organizing work

Tasks and documents have separate folder trees. Folders support names, colors and up to 12 levels of nesting. Breadcrumbs and full paths make location visible. Create a child with New subfolder or choose a parent in the creation form. Moving a folder moves its descendants logically; ownership and cycle/depth checks run on the server.

Deleting a folder keeps its directly assigned items and children by promoting them to its parent or the top level/Inbox/Unfiled. Folder deletion is not recursive file destruction.

Descriptions

Each folder and subfolder has an optional 2,000-character description. Add it during creation, or select the folder and choose Add description / Edit description. Emptying and saving removes the description. The description appears below its path and is searchable.

Images and files

Files & media beneath the description supports multiple uploads, drag and drop, clipboard images while focused, and linking existing Documents. A folder supports 50 attachments with a 10 MB per-file upload limit. Previews for images, audio and video load on demand; files can be opened, downloaded through the shared save flow or unlinked. Missing/trashed files remain removable unavailable links.

Uploaded files and their attachment relationship are created in one transaction. Document-folder uploads are stored in that folder; task-folder uploads are stored in Unfiled. Unlinking keeps the file in Documents. Folder deletion also preserves library files.

Uploading into a new folder

The document upload panel includes Save in folder -> New folder. Enter a name, description and parent, then Create & select folder. The destination changes without clearing the chosen file, name or tags. Finish or cancel folder creation before uploading. A created folder remains if the upload is later cancelled or fails.

Sources: backend/folderHierarchy.js; backend/folderAttachments.js; FolderHierarchyControls.jsx; FolderDescription.jsx; FolderAttachments.jsx; UploadFolderPicker.jsx.

CHAPTER 10

Calendar: a unified view of several sources

What appears

Month and agenda views combine task deadlines, imported Canvas assignments/events, personal appointments, date markers, sports entries and public TXST events. The calendar is a projection of several record types, not one copied list. Opening a task deadline returns to its original task. Undated tasks are shown separately until assigned a date.

Search and source/completion filters help narrow the view. Today, month navigation and agenda selection make it usable beyond the current day. Personal appointments support start/end times or all-day ranges, locations, notes and completion; markers represent dates or inclusive date ranges.

Connected records

Personal entries can link tasks and existing documents or upload supporting media. Save commits the links with the event. Cancelling a form discards unsaved links but can leave uploaded files in Documents. Deleting/unlinking an event preserves its linked tasks and documents.

Dates are not all the same

Timed events represent instants displayed in the chosen account time zone. All-day dates represent calendar dates and must not shift to the previous/next day because of a UTC conversion. This distinction also matters for multi-day ranges and imported ICS end dates.

Remove versus delete

Remove this event targets exactly one event ID. Excluding a task deadline does not delete its task or modify Canvas. Excluding a public event does not alter the source. Deleting a personal calendar record is a different operation. Matching-name removal is a separate reviewed action; similar titles alone do not trigger bulk removal.

Interview explanation: "I preserve source identity and apply visibility separately. That prevents a calendar-only action from destroying the underlying academic record."

Sources: CalendarPage.jsx; CalendarAttachments.jsx; backend/calendarEntries.js; backend/calendarLinks.js; frontend/src/lib/calendar.js.

CHAPTER 11

Calendar: recurrence, colors, categories and undo

Personal organization

Users can create named categories, choose category colors and filter by category. Individual color overrides remain available. Categories are account-owned, validated against events and cannot be deleted while still in use. A category is organization, not an external event source.

Recurring meetings

Repeat controls can generate grouped meetings at minute, hour, day, week or month intervals, including chosen weekdays. The implementation stores a finite set of entries with a shared series ID and rule metadata. Creation accepts 2-500 meetings; it is not an infinite scheduler. Series operations can update shared details or remove the group; per-event actions still target an individual item unless the group action is selected.

The creation batch is transactional. Linked task/document ownership and category validity are checked. Time-zone and calendar arithmetic are important around daylight-saving transitions and month boundaries; review the preview before saving.

Similar-name removal

Remove similar opens a reviewed selection across loaded events. Exact matching normalizes case, accents and punctuation. Similar matching uses shared words/token overlap rather than an AI classifier. Uncheck matches to keep. It operates on loaded IDs, not an eternal rule that removes every future event with the same words.

Undo boundaries

Undo and Ctrl/Cmd+Z outside text fields reverse the latest supported saved calendar change: creation, edits, deletion, series/import batches, visibility and related settings changes. History persists in MongoDB and rejects conflicting newer changes rather than overwriting them. Calendar undo consumes an entry and has no redo. Task edits outside calendar history, source changes at TXST, unsaved text and account reset are not universal undoable operations.

Sources: CalendarRecurrence.jsx; CalendarCategories.jsx; CalendarUndo.jsx; backend/calendarSeries.js; backend/calendarHistory.js; backend/calendarCategories.js.

CHAPTER 12

Public TXST events and the scraper

User experience

Opening Calendar starts loading upcoming/ongoing public TXST events alongside personal work. Refresh TXST requests another collection; progress and errors remain visible. Filters and an event inspector expose names, dates, descriptions, locations, organizations, categories, themes, benefits and source/RSVP links when available.

Public events are not inserted as personal tasks or appointments. An account can hide individual IDs, remove similar loaded events, hide all current/future campus events, or restore exclusions. Those preferences survive refreshes without changing the public source.

Collection design

The backend scraper requests public CampusLabs discovery search pages using skip/take pagination, deduplicates IDs, then retrieves details sequentially. It uses delays, timeouts and transient-error retries. JSON preserves normalized data plus raw listing/detail fields; CSV offers a flatter export. A partial collection contains errors and a complete flag instead of pretending every result was retrieved.

Calendar requests share an in-process public cache. An open calendar can trigger refresh after an hour; manual refresh has a cooldown. Partial results can retain prior data, so stale events are possible until a complete refresh. A restart clears this cache, and multiple backend processes do not automatically share it.

Interview caveat

"All events" means all publicly returned events within the selected search, not private, restricted, unpublished or deleted university records. Website endpoints can change. The app does not bypass login restrictions, RSVP on behalf of the user, or guarantee live upstream compatibility.

CLI examples: npm run scrape:txst in backend; optional --from, --to, --out and --csv arguments. Pass an explicit time-zone offset when a UTC midnight date boundary is not intended.

Sources: backend/scrapeTxstEvents.js; backend/txstCalendar.js; backend/TXST_EVENTS.md; frontend/src/hooks/useTxstEvents.js.

CHAPTER 13

Sports and F1 season import

What the feature does

Calendar supports manual sports entries with sport, league, team/opponent and normal appointment details. F1 has a dedicated schedule importer, rather than a promise to scrape every sport automatically.

Choose a season, preview the returned schedule, and select session types. Controls distinguish free practices, qualifying, sprint qualifying, sprints and Grands Prix. Only buttons make it easy to select one group. Individual rows can also be included or excluded before importing. Choose an optional calendar category for the resulting records.

How it avoids duplicates

The backend normalizes schedule sessions into stable external IDs. Re-import skips already imported sessions rather than creating duplicate appointments. The importer can categorize selected existing entries without overwriting the user's edited times/details. A batch import and its supported changes participate in calendar undo.

Data quality

The F1 schedule comes from Jolpica's public data. Sessions with known start times use those timestamps; finish times may be estimates. Missing times can appear as all-day placeholders. Public data and race schedules can change. Imports are snapshots, not a subscribed live calendar that continuously rewrites the user's events.

Interview answer

"The interesting part is not displaying a list of races. It is normalization and idempotency: identifying the same session on a later import, separating session types, handling missing timing data and preserving user edits. I make uncertainty visible instead of inventing an exact end time."

Suggested demo: preview a season, choose only Grands Prix, assign a Sports category, import, then demonstrate that another import reports existing entries. Only show that flow after verifying the live endpoint yourself.

Sources: F1SeasonImport.jsx; backend/f1Calendar.js; CalendarPage.jsx; backend/calendarHistory.js.

CHAPTER 14

Canvas synchronization and ICS import

Canvas connection

The Canvas section accepts connection information and supports manual synchronization of assignments and calendar events. Synced data feeds task lists and the unified calendar. Source IDs/types distinguish assignments from events. Updates refresh source titles/dates while preserving local task state, notes, folder choices and dependency relationships.

Pagination is part of correctness. The backend follows same-origin pagination links rather than assuming one response contains all results. Calendar contexts are batched instead of sent in an unbounded request. Partial failures are reported so one successful course does not hide another failed course.

ICS files

Users can import an exported calendar file. The parser handles folded lines, UID-based updates, UTC and named-zone timestamps, and all-day end dates. Invalid dates are skipped, not replaced by today. Floating times without a zone use the implementation's UTC fallback. Existing ICS recurrence rules are not expanded into every occurrence locally.

Limits and privacy

These are imports/syncs into the application, not two-way editing of Canvas. A status change in the app should not be described as completing an assignment in Canvas. Credentials stay out of AI workspace context, but Canvas credentials and their storage still deserve a separate security review; do not assume they have the same encryption path as saved Gemini keys.

Interview answer: "I separate source identity from local state so an import does not erase user work. I also treat a partially fetched collection as partial, not empty. A production extension would add a scheduled job queue, monitored retries and a clear conflict policy for source changes."

Sources: frontend/src/App.jsx; backend/canvasPagination.js; backend/canvasTasks.js; backend/calendarICS.js; backend/server.js.

CHAPTER 15

Document library and supported formats

Library features

Upload files up to 10 MB, rename the display name, add tags, organize folders/subfolders, search and sort. Select individual or multiple records to move or send to Trash. Drag documents onto folder destinations. Restore from Trash or use the library's deletion undo. Linked tasks are visible from file rows.

The legacy PDF collection stores multiple file kinds. Opening a document routes by extension/MIME type instead of forcing every file into a PDF renderer. Content saves preserve an uploaded original backup; original plus edited content must fit within the 15 MB storage rule. Large or unsupported content remains downloadable.

Format behavior

- PDF: rendered pages, annotation tools, page operations, selected source-text replacement and export with annotations.
- Images: converted to an editing surface; annotated output can be exported as an edited image.
- TXT/code/Markdown: UTF-8 text editing; notes-tagged text and Markdown use the writing studio.
- DOCX: editable text with original package-based serialization, retaining supported document formatting and embedded package content. The text editor is not a full Word layout preview.
- CSV/TSV: quoted-field-aware cell grid; larger cell sets can fall back to text editing.
- XLSX: select worksheets, edit values/formulas, add visible rows/columns. Excel recalculates formulas on open; this is not an in-browser Excel formula engine.
- Browser-supported audio/video: playback, download and replacement.
- Other formats: storage, download and same-format replacement; no arbitrary native editor.

Interview answer: "Format dispatch gives each file a realistic capability set. Preserving a DOCX package is different from reconstructing a Word document, and keeping an original backup provides an escape hatch when editing is limited."

Sources: DocumentLibrary.jsx; DocumentEditor.jsx; frontend/src/lib/documentFormats.js; frontend/src/lib/wordTemplateEditor.js.

CHAPTER 16

Typed notes and the writing studio

Creation and structure

Documents -> New notes offers Blank, Lecture, Cornell study, Meeting, Project brief and Daily journal templates. New notes use editable Markdown. Existing notes-tagged text and Markdown also open in the studio. Templates provide headings, prompts, tables and checklists; they do not claim to automatically know the user's course or meeting.

Writing features

- Write, Split and Read views; live formatted Markdown with mathematical notation.
- Headings, bold, italic, strikethrough, inline code, bullet/numbered lists, checklists, quotations/callouts, links, tables, dividers and display equations.
- Heading outline with navigation, word/character counts, approximate reading time and checklist completion counts.
- Literal find/replace-all with optional case matching; bounded undo/redo history.
- Enter continues a list; Shift+Enter inserts a plain newline. Tab inserts two spaces; Shift+Tab can leave the field.
- Modern, Book and Mono display fonts and four text sizes; preferred view/font/size are remembered locally.
- Focus mode removes surrounding clutter. Save stays available; Escape dismisses Find or exits Focus.

Saving and exports

Manual Save persists content; optional Autosave saves after a writing pause. Session drafts support recovery when available, and unsaved content triggers leave-page protection. Failed autosave leaves the note unsaved and displays an error; Save can retry.

Download Markdown preserves editable source. Export PDF opens the existing converter with current text, including unsaved edits, and creates a separate downloadable PDF. Writing font/size are display preferences, not a promise that PDF export inherits those settings. The export preview has its own typography.

Interview explanation: "I chose Markdown as a durable source format with a richer editing shell. It gives portable notes and local previews without introducing arbitrary editable HTML. The tradeoff is that toolbar actions insert syntax rather than providing a complete word processor."

Sources: NoteStudio.jsx; noteTemplates.js; DocumentEditor.jsx; MarkdownPDFCreator.jsx.

CHAPTER 17

Document creation: blank, Markdown and AI

Blank handwritten notebooks

Create a notebook with 1-500 empty pages in US Letter or A4. It opens directly in the PDF editor for handwritten notes. This differs from typed Markdown notes: it is a page-based annotation workflow.

Markdown to PDF

Paste Markdown or import .md/.markdown/.txt, choose a name and A4/Letter, then inspect the preview. Headings, lists, code, tables, quotations and KaTeX mathematics render locally. Export preservation handles typed spacing and wrapped lines, with special preparation for math so fraction bars and glyphs are not naively clipped.

Long content is split into bounded conversion sections and merged into one PDF. This avoids treating a browser canvas-size limit as a user instruction to manually split every document. The result has page numbering and can be downloaded or saved into the library and opened for annotation. It is a rendered export, not a guarantee of fully selectable/vector typeset text. External images remain links. Conversion and file-size limits still apply.

Create with AI

Describe the desired document. Gemini returns a draft title/text for review. After editing the draft, create a PDF that opens in the editor. Generated text is placed as editable annotations on base pages, so use the PDF editor's Download to include it in the finished file. Plain-text generation does not constitute full LaTeX typesetting or image generation.

The generation flow uses the configured AI key and generation permissions. The basic document creator sends the supplied description, not the entire document library automatically. Review facts and unsupported characters before relying on the result.

Interview distinction: "The AI drafts content; local code creates and paginates the editing artifact. Markdown conversion is a separate deterministic local workflow and does not require spending model tokens."

Sources: DocumentLibrary.jsx; AIDocumentCreator.jsx; MarkdownPDFCreator.jsx; markdownPdfExport.js; backend/generatedDocuments.js.

CHAPTER 18

PDF editor: annotation fundamentals

Tool modes

Select text is a reading/copying mode. Pen draws freehand ink. Highlight adds highlighter strokes. Text inserts or edits supported text objects. Move selects and repositions inserted objects. Erase affects annotations, not original page content.

Annotation controls include named/custom colors, numeric and slider sizes, and pen pressure where supported. Pen widths span 1-40 pt and text controls 6-144 pt. Toolbars group settings rather than showing every control continuously. Undo/redo apply to annotation changes; Save persists them and Download burns visible edits into an exported PDF or edited image.

Eraser choices

Whole stroke removes matching strokes. Precision removes intersecting portions of vector strokes. Objects removes whole text/image annotation objects. Shape, size and pen/highlighter filters let the user choose the target. A footprint indicates the active area. Erasing is undoable and does not remove printed PDF content.

Straight-line assistance

A long, sufficiently straight pen stroke can snap to a line at release. Keep / Undo lets the user retain or revert it; immediate Ctrl+Z first restores the original freehand points. The detector rejects short strokes, bends and backtracking and does not change highlighter strokes. It can be disabled.

Images, snippets and inserts

Insert includes files/images, PDF pages, shapes, image tables and stamps. Tables accept cell entry or pasted spreadsheet cells up to 12 rows by 8 columns, then insert as an image. Stamps have editable text and presets such as APPROVED, DRAFT, REVIEWED and CONFIDENTIAL. Shapes and other inserts can be moved and resized. PDF merge accepts multiple files, import ranges and before/after/start/end placement. Copy region captures a selected page region as a snippet; paste it elsewhere and move/resize it. This transfers rendered content, not necessarily the original editable source objects. The selection/move tools and keyboard nudges help place inserts precisely.

Interview answer: "I keep a semantic annotation model separate from the base document. That supports undo and editing without mutating the PDF after every pointer sample. Export is the point where annotations are composed into a portable artifact."

Sources: PDFEditor.jsx; PDFInsertPanel.jsx; PDFEraserControls.jsx; pdfEraser.js; pdfStraightLine.js; insertGeometry.js.

CHAPTER 19

PDF text: editing, fonts and copying

Editing existing selectable text

Choose Text and click a supported source text run. The editor extracts text and style and creates a replacement annotation. Detection considers font family, size, bold/italic, color, opacity, spacing, baseline and horizontal scaling. Available embedded font programs can be reused for preview/export; the catalog and same-page peer information help matching when direct metadata is limited.

Leaving text unchanged preserves the original rendering. Unsupported characters and missing/custom subset fonts can require an approximation or stop export rather than silently substitute every glyph. Independently skewed, vertical or image-only text is not equivalent to ordinary horizontal selectable text.

A crucial security distinction

Source text is visually covered and replaced. It is not necessarily removed from the PDF content stream. This is editing by overlay, not secure redaction. A white cover also cannot reconstruct arbitrary photographs or colored backgrounds behind the old text. Never claim otherwise in an interview.

Copying text

Select text mode exposes native selection without drawing. The text layer includes different fonts, sizes and positions plus relevant saved text annotations, and suppresses covered source runs for copying. Copy all text extracts across pages, including pages not currently rendered. Join wrapped lines groups soft line wraps into coherent paragraphs while preserving useful structure such as lists, headings, contact lines and code-like content.

Practical limitations

PDFs store positioned glyphs and drawing commands, not necessarily semantic paragraphs. Character geometry and reading order can be ambiguous, particularly in columns or complex layouts. Scanned pages need OCR; selection improvements do not create text that does not exist. Always review copied output from difficult documents.

Sources: pdfExistingText.js; pdfTextStyle.js; pdfFontCatalog.js; pdfPeerFont.js; PDFSelectableLayer.jsx; pdfCopyText.js.

CHAPTER 20

PDF pages, views and productivity tools

Page organization

Navigate with the page field, thumbnails or previous/next arrows, including fullscreen arrows. Insert a blank page after the current page, import page ranges from another PDF, or duplicate supported pages through Insert -> Pages. Drag thumbnails to reorder. Delete a page when more than one remains. Page mutations remap annotation page numbers to preserve their association with content.

Page operations are different from an ink gesture. They save document structure and may clear the annotation undo stack; do not promise every deleted/reordered page can be restored with the normal ink Undo button. A clean copy creates a separate document without editor annotations. Changes already burned into the underlying PDF are not removable merely by clearing the annotation array.

Reading and comparison

Single-page and continuous views support different reading habits. Continuous view activates a page using viewport visibility and waits around active gestures/text edits to reduce flicker. A side page can show a reference page while another page is edited. Fit width, zoom, view rotation and page navigation remain available.

Interface customization

Fullscreen reduces surrounding UI while keeping annotation tools, navigation and access to the assistant. Shrink/Expand changes toolbar density. Toolbar position can be Automatic, Top, Bottom, Left or Right; side positions are vertical. Dark pages invert the displayed page/annotations and thumbnails; saved/exported colors remain original. Photos also invert in this view.

Small utilities

Clock visibility, stopwatch and countdown timer support focused work. They are interface utilities, not time-tracking records or server-scheduled reminders. Check their visible controls for start/pause/reset behavior; a closed or suspended browser is not a guaranteed alarm service.

Sources: PDFEditorChrome.jsx; PDFEditor.jsx; backend/pdfPages.js; PDFEditorClock.jsx; PDFStopwatch.jsx; PDFCountdown.jsx.

CHAPTER 21

Pen, touch, mouse and rendering performance

Input arbitration

Scroll and Draw are explicit finger modes. One-finger behavior follows that mode; two fingers can pan and pinch. Pen contact takes priority over touch. A second contact can cancel a tentative finger stroke before starting navigation. The implementation uses browser pointer types and gesture ownership, not a machine-learning palm classifier.

Native stylus button events can map the main side button, secondary button and eraser tip to supported tools/actions. The settings page provides graphical controls and remembered preferences. Browser/hardware support varies: a web app cannot receive a hardware button event that the OS or browser never exposes.

Pinch zoom anchors around the finger center, uses a composited preview during the gesture and commits a final render scale. Panning can coast using recent velocity with elapsed-time friction, bounded speed/duration and scroll-edge limits. New input cancels momentum; reduced-motion preferences disable added glide. Native wheel/trackpad scrolling should not receive a second inertia effect. Mouse interactions still support drawing, selection and the applicable drag/navigation controls.

Avoiding per-sample React work

Completed ink and live ink use canvas layers instead of one SVG/React element per pressure segment. Live strokes paint incrementally; completed strokes append where possible. Eraser hit tests skip distant strokes using cached bounds. Continuous pages render lazily, canvas backing resolution is bounded, and a worker can serialize annotation JSON. Autosave waits for active gestures to pause and skips stale queued snapshots.

Attribution and honesty

Pointer-input helpers were adapted from Excalidraw v0.18.0 under MIT; the repository includes source notes and a license. This is not embedding the entire Excalidraw editor or implementing hardware palm detection. Say the architecture aims to reduce work and bound resources; do not claim zero lag or a measured speedup without profiling.

Sources: usePDFTouchInput.js; PDFInkLayer.jsx; annotationSerializer.js; PDFStylusSettings.jsx; frontend/src/lib/vendor/excalidraw/SOURCE.md.

CHAPTER 22

Mathematics, calculator, function scanning and graphs

Local calculation

The PDF editor has a calculator and region-based tools. Drag a box around a printed arithmetic expression to extract its text and evaluate it locally. Supported syntax includes common arithmetic, parentheses, powers, percentages, roots and constants. Recognition normalizes common PDF symbols into the parser's supported syntax.

Scan function -> calculate lets the user select a function, review its recognized expression, insert an input value and compute the result. This is not general symbolic algebra or proof generation.

Recognition modes

Text only uses local text extraction. Auto attempts local reading and can fall back to Gemini when needed. Handwriting / AI uses the selected image crop. The crop includes relevant visible annotations rather than sending an unrelated whole workspace screenshot. AI access and recognition permission must allow it.

AI is used to transcribe into supported math syntax; deterministic local code evaluates the result. A visibly incorrect transcription can still produce a mathematically correct answer to the wrong expression, so review is part of the workflow.

Graphing

Graph selected region or type an explicit function of x. Adjust X/Y limits, inspect the plot, insert it as a movable image annotation or download a PNG. Supported functions include common trigonometric, logarithmic, exponential and rounding functions. Trigonometry uses radians, log is base 10 and ln is natural logarithm.

Plots use finite sampling. They can miss narrow features or behave poorly around discontinuities; implicit relations and arbitrary multivariable equations must be rewritten. This is not a complete computer algebra system.

Interview answer: "I separate recognition from computation. That gives a useful AI fallback for handwriting without delegating every arithmetic result to a language model. The parser restricts the accepted expression language rather than executing arbitrary JavaScript."

Sources: PDFCalculator.jsx; PDFGraph.jsx; calculator.js; pdfCalculation.js; pdfGraph.js; mathSelectionImage.js.

CHAPTER 23

OneNote imports and preserving handwriting

Entry points

Upload a .one section or .onepkg notebook through the normal document upload, or open Import OneNote. Review source items, select the pages/sections to keep, inspect the preview and choose Save & open notes. The destination folder and tags are retained. The result is a PDF for the platform editor.

Native path

The preferred path asks desktop OneNote for Windows on the backend machine to publish source pages as PDFs. This preserves OneNote's native rendering of handwriting and annotated printouts. Imports serialize access to the shared application and retry temporary COM busy conditions. If page discovery fails, a whole-section native export can preserve the section's PDF sheets.

The native preview can report source ink counts when available and offers Download original OneNote PDF for comparison. Saving a single native result preserves its exact bytes; combining selections retains all PDF sheets.

Compatibility path

An explicit compatibility option uses the separately installed Python converter. It is cross-platform in intent but can lose unsupported ink, objects or layout. SVG ink is dimensioned/rasterized for conversion. The app should not silently switch from failed native export to a lossy result.

Limits and troubleshooting

Uploads are 10 MB, source pages are bounded, and resulting PDFs must fit the library limit. Desktop OneNote must open normally under the backend Windows account, complete sign-in/setup and unlock protected sections. Keep original files; flattened imported ink cannot be erased as individual platform strokes. New annotations are editable, but source OneNote objects are not.

Interview answer: "A notebook parser was insufficient for annotation fidelity, so the preferred route uses the original application's renderer. That improves visual preservation but introduces a Windows/desktop dependency. Exporting PDF from OneNote and uploading it is an explicit alternative."

Sources: OneNoteImport.jsx; backend/oneNoteImport.js; backend/scripts/export_onenote.ps1; backend/scripts/import_onenote.py; backend/requirements-onenote.txt.

CHAPTER 24

AI assistant: conversations and user experience

Access and controls

Open the dedicated assistant page or the floating panel; Alt+J toggles/focuses it and Alt+8 opens its page. The panel can be moved, resized, filled to available space or reset. Page and panel dimensions have separate local preferences. Fullscreen PDF editing can expose the assistant beside the document.

Conversation management

Saved account-owned chats support automatic titles, rename, pin, archive/restore, search, pagination, private direct links, Markdown export and permanent deletion. Copy messages or code blocks. Edit an earlier question in a branch or continue from an earlier answer without overwriting the original conversation. Regenerate replaces an answer only after success; failed regeneration preserves it. Continue helps with truncated replies, and Jump to latest helps when reading history.

Messages render Markdown, tables, code and KaTeX inline/display math, with sources and context summaries. Prompt starters and Concise/Balanced/Detailed response settings help guide use. Attachment choices and settings persist per conversation. Star a saved question to capture it and its available answer in Study stars.

History is bounded

The server supplies saved history, not an arbitrary browser-authored transcript. Conversations have a message cap, while only a bounded recent slice is sent to the model. Older messages remain viewable; a long chat does not mean the provider sees all of it on every request. Generation leases and revision checks reduce overlapping replies from multiple sessions.

Interview phrasing

"The assistant is integrated into the workflow, but it is not an unrestricted autonomous agent. It answers with selected context and can prepare a narrow set of supported edits for the currently open editor when authorized. It cannot arbitrarily mutate tasks or browse the entire internet as a general tool."

Sources: AssistantChat.jsx; AssistantMarkdown.jsx; useAssistantResize.js; backend/assistantConversations.js; backend/assistant.js.

CHAPTER 25

AI context, credentials, access and diagnostics

Explicit context

Per-chat switches choose structured workspace information, visible page text and the open file. Users can select explicit file attachments. Supported PDFs/media may be sent as provider inline inputs; text/DOCX/workbooks use extraction with bounds. Visible-screen context is text, not an automatic screenshot of the whole computer. Unsupported content gets a notice instead of an invented extraction.

Account-level controls

AI access controls can disable AI globally or restrict profile, tasks/folders, calendar, library metadata, resumes/letters, notes, other document bodies, screen text, history, live editor reading, editor writes, generation and recognition. Backend enforcement overrides per-chat switches. Restricting categories can suppress screen/history context to avoid leaking protected information through a different channel. It cannot recall data already sent, remove historical provider copies or prevent a user from deliberately typing private information into a prompt.

Saved credentials

Personal Gemini keys are encrypted server-side with AES-256-GCM and account-bound associated data; a stable server secret derives the encryption key. The browser receives masked status, not the stored raw key. A configured server fallback can apply when a user has no personal key. Saving a key does not prove quota, billing or model availability.

Failure handling

The backend bounds request concurrency, context and rate. Timeouts abort requests; it does not indiscriminately retry provider calls. Diagnostics distinguish authentication, permission, model, quota, throttling, safety/context, network and service failures. Trace IDs, stage, timings, safe provider details and retry countdowns make errors actionable. A deliberate connection test consumes provider usage; a configuration/status refresh need not call Gemini.

Interview answer: "AI safety here is primarily capability scoping, data minimization and validated output, not a claim that prompt instructions make a model perfectly trustworthy. Provider policy and availability remain external dependencies."

Sources: backend/aiAccess.js; backend/assistantCredentials.js; backend/assistantDiagnostics.js; AccountAIAccess.jsx; AssistantDiagnostics.jsx.

CHAPTER 26

AI document editing and PDF cleanup

Editing the open document

Enable Open file context and allow edits, then request a specific change. The browser's editor bridge exposes only the mounted editor's supported live text IDs and revision. Supported actions include replacing supplied text, adding a PDF text annotation on the current page and navigating pages. Plain-text notes can accept an entire-document text replacement. Closed attachments, Word files and spreadsheets are not arbitrary writable targets through this bridge.

The provider prepares an editor-actions JSON block. Backend validation checks action type, count, IDs, text length, coordinates and page. The browser rejects stale results when the document/page/version has changed. Reopening a saved chat does not replay actions. A PDF batch becomes one annotation undo step and uses annotation persistence; text notes still require saving or their enabled autosave.

Clean up PDF

The cleanup tool accepts user comments, extracts supported text from the current page or bounded document scope, and asks for proposed corrections. The user reviews individual proposals and can select/edit them before applying. Supported corrections become text edits in one undo batch; it does not magically repair arbitrary handwriting, diagrams or document semantics.

Why both layers validate

"The server checks what the model is allowed to propose, and the browser checks whether the target is still current. That handles both malformed output and the race where I keep editing while a request is in flight. The language model is not given direct database or filesystem access."

Restrictions remain real: PDF replacements are visual overlays, font coverage can fail, scans may lack text, and context is bounded. Do not describe this as full autonomous document editing, secure redaction or guaranteed correctness.

Sources: frontend/src/lib/editorBridge.js; backend/assistantEditor.js; PDFCleanup.jsx; backend/pdfCleanup.js; DocumentEditor.jsx.

CHAPTER 27

Jobs: links, screenshots, PDFs and local extraction

Saved job records

The Jobs page stores a posting's title, company, location, type, salary, dates, URL, description, requirements, responsibilities and benefits. Users review fields before saving and can return to the original posting. Saved roles provide context to cover-letter and resume generation.

Import by link

The backend retrieves public posting pages and uses structured job data and site-specific/general parsing where available. Outbound retrieval validates public URLs, addresses, redirects, timeouts and body size. Handshake receives special handling for publicly available posting data and a pasted-content fallback when the page depends on login or client-side content.

Universal does not mean unrestricted: the server cannot access a private authenticated session merely because the user can see that page in another tab. The app should ask for pasted text or a captured document rather than bypass access restrictions.

Import from files

Upload an image or PDF of the posting. Local mode uses PDF text/layout extraction or browser OCR for raster content, then heuristics to classify header fields and sections. The layout-aware parser helps separate title, company and location from a long description. Manual correction remains available. AI mode is an explicit alternative and can consume tokens; the local extractor is deterministic/OCR-based, not a privately hosted language model.

Interview explanation

"I try cheap structured extraction first, retain raw evidence, and let the user correct the result. The difficult part is recovering semantic fields from arbitrary page layout, not just extracting text. A bigger version would use labeled evaluation sets and measure per-field precision/recall before claiming broad accuracy."

No claim of perfect imports is warranted. A screenshot can omit context, OCR can confuse symbols and columns, and different job boards can change their markup.

Sources: JobsPage.jsx; JobFileImport.jsx; localJobFile.js; frontend/src/lib/jobParsing/; backend/jobs.js; backend/jobFileImport.js; backend/jobExtraction.js.

CHAPTER 28

Cover letters: reference voice and template fidelity

Workflow

Upload and store one or multiple cover-letter files in the career section. Select a template/reference letter and a saved role or supplied job details. The generator uses source material to propose role-specific replacements, with existing wording and factual evidence prioritized. Review the generated draft in a continuous text box rather than disconnected sentence chips.

Previous letters guide voice and phrases as well as layout. The workflow reports evidence, warnings and verified phrase reuse so the user can judge whether the result sounds like them. Existing documents are evidence, not permission to invent new achievements or copy every irrelevant sentence into the output.

Preserve the original artifact

The generation model supplies structured text replacements, not the final PDF/DOCX binary. Local/template logic places reviewed replacements into supported original regions. PDF edits reuse compatible font/layout slots where possible. DOCX edits preserve the original ZIP package and XML structure rather than rebuilding everything with a generic font. If a replacement does not fit or the font lacks a glyph, the app should expose the limitation instead of silently shrinking or substituting everything.

Save and error review

Save a separate copy and open the editor, leaving the reference intact. Review/acknowledgement gates and descriptive warnings help avoid treating an AI draft as a final application. Expandable errors include stage, category, safe provider/application details, HTTP status, trace ID and suggested next step. Compact prompts and bounded references reduce unnecessary context; provider latency can still vary.

Interview answer: "The model handles language tailoring; deterministic code retains format ownership. I prioritized matching the user's source phrasing and supported facts rather than optimizing for a generic polished AI voice."

Sources: CoverLettersPage.jsx; CoverLetterGenerator.jsx; CoverLetterError.jsx; backend/coverLetterPrompt.js; backend/coverLetterGeneration.js; coverLetterPdfLayout.js; wordTemplateEditor.js.

CHAPTER 29

Resumes and company-specific tailoring

Resume storage and generation

The resume section stores PDF/DOCX resumes that are also accessible in Documents. Choose a source template, relevant previous resumes and a role. Resume-specific instructions prioritize source-supported skills, projects and experience while preserving identity, employment/education chronology and official titles. It is not a license to invent qualifications for a job description.

The resume generator shares document-generation infrastructure with letters but uses a different prompt and purpose. References for resumes come from resume material. Review replacements and warnings before saving a separate copy.

Annotated templates

The workflow accounts for saved editor text annotations as well as source PDF text. Covered original runs should not be duplicated into the prompt as if still visible. Saved annotation-backed regions can be updated while retaining other ink/images/highlights. A template version check prevents a draft from being applied against a newer source file. Handwriting/images retained as visual content do not become applicant facts unless actually interpreted by a supported path.

Company website research

Both resume and letter generation can use the employer's official public website. Confirm/enter its URL; known job-board/ATS URLs are not substitutes for the official company site. Bounded same-host public pages provide business/product context, with source links and excerpts in review. Employer context helps prioritize relevant existing projects, not fabricate experience or affiliation.

Public pages are untrusted input. Retrieval is bounded, validated and separated from applicant documents; the app does not send the user's resume to the employer website. Website failure falls back with a warning to role and source documents. Cleanup-only letter mode does not need research.

Interview answer: "I distinguish employer facts from applicant facts. Research can change relevance, but it must not change what the candidate actually did."

Sources: backend/resumePrompt.js; backend/companyResearch.js; backend/coverLetterGeneration.js; tailoringTemplateRuns.js; ResumesPage.jsx.

CHAPTER 30

Custom workspace and task visualization

A workspace of tools

Workspace provides saved workflow tabs containing draggable/resizable windows. A window can hold tool tabs for Tasks, Agenda, Documents, Task map, Notes, Focus timer and the assistant entry point. Users can group/split tools, rename tabs, minimize/maximize windows, tile, choose templates, adjust density/grid/snapping, lock layout and use fullscreen mode.

Planning, Research and Minimal templates are starting arrangements, not copies of the user's underlying data. Scratchpad notes are workspace content, separate from standalone document notes. Export/import JSON transfers supported layout and workflow-note state; it does not export every task/document binary. A focus timer is local runtime behavior, not a durable scheduled job.

Layout and notes save to the account with revisions. A browser draft helps recover failed/offline saves. Conflict handling offers reload or save-this-version choices rather than silently overwriting a newer layout. Layout undo/redo is distinct from calendar or document undo.

Visualization

The task map groups by folder/status or uses planned sequence. It shows actual task dependencies and document relationships. Search, status/folder/overdue filters, pan/zoom, fit/reset, minimap and an inspector support exploration. Users can open tasks/documents, edit and advance status. Completion, workload and deadline charts are derived from the same records.

Dragged map positions are temporary view adjustments, not persistent changes to task order. Filters can hide graph edges to excluded nodes without deleting the relationships.

Interview answer: "The workspace composes tools around shared records. The graph is another projection of that same model, so I avoid maintaining a second task database just to display a visualization."

Sources: WorkflowWorkspace.jsx; WorkspaceWindow.jsx; WorkflowTools.jsx; useWorkspaceLayout.js; backend/workspaceSettings.js; TaskVisualization.jsx; taskVisualization.js.

CHAPTER 31

Study stars and learning workflows

Saving material

Tools -> Study stars is a dedicated collection for questions and topics. A saved user question in the AI assistant has a star control. Starring captures the question, its available following answer and a source conversation reference. It is not automatically a synchronized mirror of all future revisions to that answer.

Users can also create manual questions/topics with a subject, answer/study material and personal notes. Edit or unstar items, search their contents, filter by Questions/Topics/To review/Studied and mark a review status. Reveal answer/material supports active recall instead of displaying everything immediately. A source conversation link returns to the saved chat when still available.

Ownership and persistence

Study records are account-owned and stored in MongoDB. When starring a saved conversation question, the backend loads the owning conversation and message rather than trusting an arbitrary client-supplied answer. A source key prevents repeated stars for the same source question from creating duplicates. Study material participates in global search and account reset.

An example learning flow

Create lecture notes, attach a source PDF, ask the assistant to explain a concept, star the difficult question, then revisit it in Study stars and reveal the answer only after attempting recall. Manual topic entries can organize material that did not originate in the assistant.

Interview answer: "I wanted generated explanations to become reusable study material rather than disappear in chat history. The current feature is a review collection with status tracking, not a spaced-repetition algorithm or an adaptive learning model. Scheduling reviews would be a separate enhancement."

Sources: StudyPage.jsx; backend/study.js; AssistantChat.jsx; frontend/src/lib/study.js.

CHAPTER 32

Account, authentication and security boundaries

Account features

Account includes profile/activity information, time-zone selection, action-confirmation preferences, task-column visibility, AI credentials and access controls, connected accounts and clean account reset. Time-zone settings can follow the device, select an IANA zone, search cities or request an IP-based suggestion. These optional lookups depend on external services; VPNs can affect IP location.

Authentication

Registration/login use bcrypt password hashing and signed JWTs. The implementation issues seven-day tokens and the frontend persists authentication state in localStorage. Protected routes verify the token and query by user ownership. File IDs alone are not authorization.

The existing CORS and token-storage choices should not be presented as a completed production security review. localStorage tokens are exposed to successful script injection. Potential improvements include a reviewed cookie/token strategy, tighter origins, stronger upload/content policies, account abuse controls and dependency maintenance.

Connected identities

GitHub/LinkedIn connections link a provider profile to the current account; they do not replace application login or automatically import repositories, contacts or social activity. Authorization uses expiring single-use state tied to a browser cookie; GitHub also uses PKCE. Provider tokens fetch profile information and are discarded. Disconnect invalidates pending attempts and removes the local link; provider-side revocation is separate.

Destructive account reset

Reset requires the current password and RESET confirmation, uses a transaction, and clears application data including recovery/history while preserving login identity. It is not undoable. Per-process write guards must be coordinated before deploying multiple backend instances. Do not offer a live reset during a demo with valuable data.

Sources: AuthContext.jsx; backend/server.js; backend/accountConnections.js; backend/accountReset.js; TimeZoneSettings.jsx; ConfirmationSettings.jsx.

CHAPTER 33

Downloads, recovery and the meaning of Save

Choose a download destination

All platform export paths share a save helper: library files/originals, PDF/image output, notes, letters/resumes, graphs, OneNote native previews, chats and workspace JSON. When supported in a secure context with user activation, the native Save As picker lets the user select folder and filename. After long preparation, a ready dialog provides a fresh Choose location click.

Cancelling does not trigger a second download automatically. Write failure keeps the prepared file available for another attempt. Browser download is an explicit alternative. Browsers without this API require their own Ask where to save each file setting; the site cannot force an arbitrary local path. Fullscreen editors use a top-layer save dialog.

Recovery is deliberately scoped

- Tasks: seven-day server snapshots, including bulk groups, with stable-ID restoration.
- Calendar: durable before/after history for supported calendar mutations, conflict-aware undo, no redo.
- Documents: Trash and restoration; content originals where supported; text/workbook session drafts.
- PDF annotations: undo/redo and annotation save, distinct from structural page operations.
- Notes: bounded edit history, drafts and optional autosave.
- Workspace: layout history, account revisions and local draft/conflict handling.

None of these promise recovery of data from before the feature existed, externally changed sources, expired snapshots, account reset, every deleted page or files already exported to another program.

Interview answer

"Save changes the app's persistent record; Download creates a local artifact. Some formats have a base file plus separately editable annotations. I make the export step explicit and use different recovery models for different data, rather than describing every action as universally undoable."

Sources: frontend/src/lib/download.js; DocumentEditor.jsx; PDFEditor.jsx; TaskDeletionUndo.jsx; backend/calendarHistory.js; useWorkspaceLayout.js.

CHAPTER 34

Separate daily-events site and QR poster

What it is

The daily-events directory is a standalone frontend project, separate from the authenticated AssignmentTracker app. It summarizes campus events for residents who scan a QR code. It uses plain HTML/CSS/JavaScript and a published events.json file; no backend or build step is required to serve that directory.

Features

- Month calendar and selected-day agenda with event details/source links.
- Previous/next month, Today, date selection and Next event navigation.
- Search, categories and a free-food filter based on source fields.
- America/Chicago date boundaries, overlap handling and midnight rollover when viewing Today.
- Dark mode by default with a remembered light-mode toggle.
- Automatic JSON reload approximately every minute, retaining the last loaded collection if reload fails.
- Manual file import for local preview; automatic reload resumes through Reload events.
- Public URL entry, locally generated QR code, printable light poster and SVG QR download.

Data freshness is not automatic scraping

The browser's minute refresh only reads events.json. The separate scraper must update/publish that file. Importing a file in one visitor's browser does not update the hosted dataset. The interface shows update time and warnings for stale/partial exports. Inspect the actual hosted file before claiming it contains current events; README passages about its included data reflect different stages of the project.

Running and explaining it

From the repository root: python -m http.server 8080 --directory daily-events. Open http://localhost:8080. Use a real public URL in a printed QR code, not localhost. Host the full directory including vendor assets and events.json.

Interview answer: "This spin-off separates public read-only distribution from the private student workspace. Static hosting is simple, but publishing fresh data remains a separate responsibility."

Sources: daily-events/README.md; daily-events/index.html; daily-events/app.js; backend/scrapeTxstEvents.js.

CHAPTER 35

A practical eight-minute demonstration

0:00-0:45 | Problem and overview

Deliver the short pitch. Open Home and show how tasks, dates and documents connect. Explain that this is one account's persisted data, not a hardcoded dashboard. Keep private account information off-screen.

0:45-2:00 | Tasks and safe recovery

Create a task, assign a folder/deadline and add a prerequisite. Drag it to another position/status. Delete a disposable task and restore it with Undo deletion. Explain the transaction-backed snapshot. If asked about hidden In Progress, show the setting without spending the entire demo customizing it.

2:00-3:00 | Calendar integration

Open the task deadline, create a personal event and link a document. Explain source identity, time zones and one-event versus similar-name removal. Demonstrate calendar undo. Use preverified campus/F1 data rather than waiting for an unreliable live import.

3:00-4:15 | Notes, folders and media

Create a lecture note template, enter a heading/list/equation, switch Split/Read, and show the outline. Add a description and a small image to its folder. Explain that attachments live in the shared document library.

4:15-5:45 | PDF engineering

Open a prepared PDF, draw one stroke, undo it, copy text and show a side page. Show a local calculation or graph. Explain that text replacement is an overlay and dark pages are view-only. Mention touch/pen input design; do not depend on unfamiliar demonstration hardware.

5:45-7:00 | AI and career flow

Show access controls and a preprepared assistant conversation. Explain a supported edit and stale-response validation. Show a saved job and a reviewed tailored document; compare source evidence rather than inventing a live result. Reveal diagnostics only if asked.

7:00-8:00 | Architecture and next steps

Use global search to return to a record. Explain React -> Express -> MongoDB and identify one honest technical limitation. End with the highest-priority improvement and why you would measure it. Keep the optional workspace, study list and daily-events QR as follow-up material.

CHAPTER 36

Interview questions: architecture and data

Why React, Express and MongoDB?

"React fits the stateful editor and composed panels. Express keeps the API straightforward, and Mongoose gives a common record layer for owner-scoped data. MongoDB accommodates evolving task/document metadata. The cost is coordinating relationships and transactions explicitly; I would revisit binary storage as files grow."

How is this more than CRUD?

"CRUD is the foundation, but the difficult parts are document geometry, gesture ownership, safe recovery, cross-entity relationships, extraction/indexing, AI context boundaries and template-preserving exports. I can show the data path for each rather than relying on a large feature list."

How do you prevent another user reading my data?

"The token establishes identity, and database filters include the owning user. Relationship writes validate that referenced documents, tasks or folders belong to that account. A valid-looking object ID does not bypass that check. I would still include negative authorization tests in a security validation plan."

Why transactions?

"A deleted task and its recovery record must commit together. The same idea applies to a calendar mutation and its undo history, or a folder attachment upload and its link. Otherwise a partial failure could lose recovery or leave inconsistent relationships. This requires a transaction-capable MongoDB deployment."

What happens with concurrent edits?

"Some workflows use revision checks or per-account transaction locks, such as chats, layouts and calendar history. Others need further concurrency review. I do not claim the entire app provides collaborative editing or identical conflict handling everywhere."

How would you scale it?

"Move binaries to object storage, run slow conversions/indexing in a job queue, paginate large collections, share public caches and rate limits across instances, and add observability. Measure memory, response times and queue delay first. These are next steps, not infrastructure already deployed."

CHAPTER 37

Interview questions: PDF, input and performance

Why are PDFs hard to edit?

"A PDF often describes positioned glyphs and graphics rather than semantic paragraphs. Replacing a sentence can require font coverage, line layout and background reconstruction. I support bounded text-run replacement and disclose that the original is visually covered, not securely redacted."

How are annotations preserved across pages?

"Annotations retain page references and document-space coordinates. Page changes remap those references. Continuous view separates passive rendered pages from the active editable overlay and waits for matching geometry. Async callbacks must not apply another page's dimensions to the active page."

How did you reduce drawing lag?

"The implementation draws live and completed ink on canvas layers instead of creating a large React/SVG tree for every pressure segment. It appends where possible, indexes bounds for erasing, caps canvas resolution and defers serialization/save work. I would substantiate the improvement with frame-time and memory profiling before giving a percentage."

Is palm rejection AI-based?

"No. It relies on pointer type, pen ownership and explicit touch modes. Two-pointer gestures are navigation. Hardware and browser events limit what can be inferred. Small input helpers are adapted from Excalidraw under MIT, with attribution; that is not a claim that I wrote hardware-level palm detection."

Why not just use a full editor library?

"A full library could reduce maintenance and provide mature behavior, but integrating a narrower annotation model gives direct control over features and exports. The decision has costs: text layout and device compatibility need substantial testing. I would reassess the boundary as the project grows."

What would you profile?

Input-to-paint latency, long tasks, stroke-count scaling, memory at high zoom/continuous view, export time, large-file extraction and worker overhead. Compare a defined device/document/stroke workload before and after changes; do not treat a visual impression as a benchmark.

CHAPTER 38

Interview questions: AI, extraction and trust

Where is AI necessary?

"It is useful for language tailoring, explanations and difficult handwriting recognition. It is unnecessary for arithmetic once the expression is parsed, ordinary search, many job imports or Markdown-to-PDF conversion. I keep those local or deterministic to reduce latency, tokens and unnecessary data sharing."

How do you reduce hallucination?

"I constrain the task, provide source evidence, bound the output schema and require review. For resumes, employer context does not authorize inventing applicant facts. For math, the model transcribes while a local evaluator computes. None of these guarantee that a model never makes an error."

Can a malicious document instruct the assistant?

"Document and website content are untrusted data. Prompts state that, but the stronger boundary is capability limitation: backend-validated actions against known IDs and browser revision checks, not arbitrary code execution. Prompt injection remains a risk that deserves adversarial evaluation."

Why can job imports still fail?

"Public HTML, OCR and PDF layout are incomplete signals. Some sites require login or render fields dynamically; screenshots may omit the title or company entirely. I provide multiple input paths and reviewable fields. The correct response to missing data is an explicit gap, not fabricated certainty."

How do you improve speed without losing quality?

"Use structured extraction first, trim irrelevant reference text, bound employer research, select an appropriate configured model and avoid blind automatic retries. Measure provider latency separately from document parsing, network retrieval and export. Faster prompts alone cannot fix provider quota or network failure."

How would you evaluate quality?

Build consented/redacted fixtures: per-field job extraction accuracy, phrase/evidence compliance, layout overflow, math transcription correctness and unsupported-action rejection. Compare local and AI modes on the same dataset. Record model/version and input conditions. Existing code and test files are not a substitute for recorded results.

CHAPTER 39

Preparing your own engineering stories

Story 1 | Making destructive operations recoverable

Situation: users can delete a task group accidentally. Task: preserve recoverability without treating deleted items as active work. Action to explain: snapshot tasks and dependencies in a separate expiring collection within a transaction, then restore stable IDs and validate surviving folders. Result to report: the implemented restoration behavior. Add a measured result only after you personally verify it.

Story 2 | Handling heavy annotation workloads

Situation: accumulating strokes makes UI work grow. Task: reduce redraw and save overhead. Action to explain: canvas layers, incremental painting, cached bounds, worker serialization and bounded rendering resolution. Result to report: architectural reduction in repeated work; supply profiling data only if you have it. Discuss quality/memory tradeoffs at high zoom.

Story 3 | Preserving a user's writing and layout

Situation: generated cover letters can sound generic and break the uploaded template. Task: tailor content while keeping evidence, voice and supported formatting. Action to explain: source phrases, structured replacement regions, template-owned serialization, font/overflow checks and human review. Result to report: the designed workflow, not a fabricated hiring outcome.

Story 4 | Recovering OneNote annotation fidelity

Situation: a compatibility parser can lose ink. Task: preserve rendered annotations. Action to explain: prefer native OneNote PDF export, keep diagnostics and explicit fallback selection, preserve all PDF sheets. Tradeoff: better native rendering introduces a desktop dependency; it cannot restore strokes absent from the source upload.

Personal preparation checklist

- Identify exactly what you built, adapted, generated or reviewed with assistance.
- Reproduce one bug and explain its cause in plain language.
- Pick a source file you can walk through without reading it line by line.
- State what was not tested and describe the verification you would perform next.
- Prepare one design decision you would change and explain the new evidence that would justify changing it.

CHAPTER 40

Limitations, testing plan and roadmap

Current boundaries worth stating

Browser capabilities vary for save pickers, stylus events, codecs and touch input. Public endpoints, company pages and AI quotas can fail or change. PDF text replacement is not redaction; fonts, scans and complex layouts are limited. Job extraction and model output require review. OneNote fidelity depends on native export and source completeness. Recurrence is finite, and undo is scoped. Some caches/locks are process-local. Large binary documents and annotation arrays pressure browser/database resources.

A meaningful validation plan

- Unit fixtures: date-only versus timed conversions, daylight-saving recurrence, parser normalization, dependency handling, page remapping and font/text logic.
- API integration: ownership failures, rollback on transaction errors, expired recovery, duplicate import, stale revisions and file limits.
- Browser/device checks: keyboard and focus behavior, large PDF scrolling, mixed pen/touch gestures, fullscreen dialogs, copy text and exported output.
- AI/extraction evaluation: known source evidence, malformed actions, prompt-injection attempts, OCR errors, per-field accuracy and formatting overflow.
- Operational checks: memory, response time, provider timeouts, logs without secrets, multi-instance coordination and restore/recovery procedures.

These are recommended checks, not checks executed for this handbook. The repository contains targeted test files, but their presence does not establish pass status.

Prioritized evolution

First: stabilize critical workflows with representative fixtures and observability. Next: split large coordinator components, centralize data fetching and formalize API schemas/conflict rules. Then: move binaries to object storage and slow jobs to a queue, with shared coordination. Finally: broaden extraction support or add review scheduling only after measuring demand and quality.

Interview closing: "I am proud of the connected workflows, but the next engineering milestone is reliability and maintainability, not adding another feature before understanding current behavior."

CHAPTER 41

Troubleshooting and demonstration recovery

App does not load

A Vite/Babel syntax error points to a source file and line; inspect that location rather than hiding the overlay. Confirm frontend dependencies and server startup. The frontend's /api proxy must reach the backend. An empty/non-JSON response often means a wrong route, stopped backend or stale running version.

Saved tasks or folders fail

Check account/session and the server's response. Transaction errors often indicate a standalone MongoDB deployment instead of a replica set/Atlas. A folder/file ID must belong to the signed-in account. Restart the backend after schema/route updates; refresh the frontend after UI updates.

Calendar is empty

Check source filters, hidden-event preferences, dates/time zone and upstream loading/error state. A minute browser refresh of the separate daily-events site is not a new scrape. Do not invent placeholder events to disguise missing source data.

AI or tailored generation fails

Expand diagnostics and identify the stage: credentials, provider request, template extraction, research, layout or saving. Verify key/model/quota yourself. Retry only when appropriate; shorten excessive input, use a clear template and check font/overflow warnings. A timeout is not automatically proof of a bad prompt.

File output looks wrong

Compare original, editor view and exported file. Confirm annotations were saved/exported, check the page count and font warnings, and distinguish native OneNote output from compatibility conversion. Dark-page display colors should not be mistaken for export colors.

Safe demo fallback

Use prepared records and local notes/PDF tools when external integrations fail. Explain the failure boundary and diagnostic rather than repeatedly calling a provider in front of the interviewer. Keep a copy of the source PDF and final reviewed artifact. Never reset the entire account as a troubleshooting shortcut during a demonstration.

CHAPTER 42

Source map and feature inventory

Frontend entry and shared infrastructure

App.jsx coordinates routes, tasks, documents and editors. AppNavigation.jsx and GlobalSearch.jsx handle navigation/search. AuthContext.jsx handles session state. documentFormats.js dispatches file kinds, download.js handles saving, editorBridge.js exposes mounted editor capabilities, and shortcuts.js defines the main shortcut registry.

Academic workflows

TaskDependencies.jsx, TaskAttachments.jsx, TaskDeletionUndo.jsx, TaskDisplaySettings.jsx and useTaskPointerDrag.js cover task relationships, evidence, recovery, layout and dragging. CalendarPage.jsx, CalendarRecurrence.jsx, CalendarCategories.jsx, CalendarAttachments.jsx, CalendarUndo.jsx and F1SeasonImport.jsx cover scheduling. FolderHierarchyControls.jsx, FolderDescription.jsx, FolderAttachments.jsx and UploadFolderPicker.jsx cover nested organization and media.

Documents and study

DocumentLibrary.jsx, DocumentEditor.jsx, NoteStudio.jsx, AIDocumentCreator.jsx, MarkdownPDFCreator.jsx and OneNoteImport.jsx are the major document entry points. PDFEditor.jsx and PDFEditorChrome.jsx coordinate PDF behavior; focused helpers cover ink, erasing, text, copy, math, fonts and gestures. StudyPage.jsx and backend/study.js implement starred material.

AI, career and workspace

AssistantChat.jsx plus backend/assistant*.js implement chat, context, diagnostics, credentials and permitted edits. CoverLetterGenerator.jsx and backend/coverLetterGeneration.js share tailoring infrastructure; resumePrompt.js, coverLetterPrompt.js and companyResearch.js separate purpose/evidence/research. JobsPage.jsx, job import modules and jobParsing helpers implement extraction. WorkflowWorkspace.jsx and TaskVisualization.jsx provide additional projections of the same records.

Backend source of truth

server.js defines models and registers routes. Feature modules own transactional behavior, parsing and external calls. Consult the endpoint and shortcut appendices that follow for a source-derived inventory. Filenames in this guide are repository-relative identifiers, not claims that the guide exercised every path.

The separate daily-events project has its own README and script assets. Vendor source notes identify adapted Excalidraw input helpers. Review licenses and attribution when presenting reused components.

APPENDIX A

Keyboard shortcut reference

These entries come from frontend/src/lib/shortcuts.js. Scope and focus determine whether a shortcut is active; text-entry exceptions and open dialogs can suppress it. Ctrl means Ctrl on Windows/Linux or Cmd on macOS where supported. Labels are registry descriptions; account confirmation preferences still apply.

Calendar

Keys	Action
Ctrl / Cmd + Z	Undo the latest saved calendar change outside text fields
N	Add a personal appointment
Shift + N	Add a date marker
T	Jump to today
[	Previous calendar month
]	Next calendar month
/	Search the calendar
Esc	Dismiss calendar form or details

Platform

Keys	Action
Alt + 9	Open the customizable workspace
Alt + 8	Open the AI assistant page
Alt + 7	Go to Calendar
Alt + 1	Go to Tasks
Alt + 2	Go to Documents
Alt + 3	Go to Canvas integration
Alt + 4	Go to Account
Alt + 5	Open the keyboard shortcuts page
Alt + 6	Go to task visualization
Ctrl / Cmd + K	Search the entire platform, including inside editors
Ctrl / Cmd + Shift + K	Open the command section
?	Open keyboard shortcuts
/	Focus the current page’s search or task input
Esc	Dismiss an open inline form or command section

Workspace

Keys	Action
Alt + N	Open the window and tool picker
Alt + T	Tile visible windows
Alt + F	Toggle full-screen workspace
Ctrl / Cmd + Z	Undo a layout change outside text fields
Ctrl / Cmd + Shift + Z	Redo a layout change outside text fields
Esc	Dismiss the tool picker/settings or exit full-screen workspace
Alt + Arrow keys / Alt + Shift + Arrow keys	Move a focused window header (Shift increases step)

Assistant

Keys	Action
Arrow keys / Shift + Arrow keys	Resize using a focused edge or corner handle (10 px steps; Shift for 40 px)
Home	Reset assistant size with a resize handle focused
Ctrl / Cmd + Shift + O	Start a new chat
Ctrl / Cmd + Shift + B	Toggle chat history
Ctrl / Cmd + Shift + F	Search saved conversations
Alt + J	Toggle the floating assistant; focus the composer on its page (works in editors)
Esc	Close the assistant panel

Visualization

Keys	Action
/	Focus task-map search
0	Fit the task map
=	Zoom into the task map
-	Zoom out of the task map
R	Reset the map layout
Esc	Clear task-map filters

Documents

Keys	Action
U	Upload a document
Shift + N	Create a folder
Ctrl / Cmd + A	Select all visible documents
M	Move selected documents
Delete	Move selected documents to Trash
R	Restore selected documents from Trash
F2	Rename the single selected document
Shift + R	Refresh the library
D	Download the single selected document

Tasks

Keys	Action
E	Edit the focused task
Space	Advance the focused task’s status
Delete	Delete the focused task (requires inline confirmation)
Ctrl / Cmd + S	Save active task or folder edits
N	Focus the new task input
Shift + N	Create a task folder

Canvas

Keys	Action
O	Choose an ICS calendar file
Ctrl / Cmd + S	Import the selected calendar file

Document editor

Keys	Action
Ctrl / Cmd + S	Save edits
Ctrl / Cmd + Shift + S	Download the edited document
Esc	Close the editor (unsaved edits are protected)

Visual editor

Keys	Action
Ctrl / Cmd + S	Save annotations
Ctrl / Cmd + Shift + S	Download with annotations
Ctrl / Cmd + Z	Undo annotation change
Ctrl / Cmd + Shift + Z	Redo annotation change
Ctrl / Cmd + Y	Redo annotation change (alternative)
V	Select the Move tool
T	Select the Text tool
P	Select the Pen tool
H	Select the Highlighter tool
E	Select the Eraser tool
I	Open or close the insert shelf
Page Up	Go to the previous page
Page Down	Go to the next page
=	Zoom in
-	Zoom out
0	Fit the document to the viewer
R	Rotate the view
Delete	Delete the selected inserted object (undoable)
Shift + =	Enlarge the selected insert
Shift + -	Shrink the selected insert
←	Move the selected insert left
→	Move the selected insert right
↑	Move the selected insert up
↓	Move the selected insert down
Esc	Close the editor and save pending annotations

Notes studio additionally handles Ctrl/Cmd+B, Ctrl/Cmd+I, Ctrl/Cmd+F, Ctrl/Cmd+Z, Ctrl/Cmd+Shift+Z and Ctrl/Cmd+Y in the writing field. Its Tab behavior differs from ordinary page navigation. See the notes chapter.

APPENDIX B

API implementation inventory

The following literal and template route declarations were extracted from backend JavaScript source. This is a navigation aid, not a tested OpenAPI contract. Parameterized registration templates retain ${...} notation; their concrete bases are explained below. Auth, body shape and capabilities must be checked in the named module. No endpoints were called.

accountConnections.js

Method	Path / registration template
GET	/api/account/connections
POST	/api/account/connections/:provider/start
GET	/api/account/connections/:provider/callback
DELETE	/api/account/connections/:provider

aiAccess.js

Method	Path / registration template
GET	/api/account/ai-access
PUT	/api/account/ai-access

assistantConversations.js

Method	Path / registration template
GET	/api/assistant/conversations
POST	/api/assistant/conversations
GET	/api/assistant/conversations/:id
POST	/api/assistant/conversations/:id/fork
PATCH	/api/assistant/conversations/:id
DELETE	/api/assistant/conversations/:id

assistantCredentials.js

Method	Path / registration template
GET	/api/account/ai-settings
PUT	/api/account/ai-settings
DELETE	/api/account/ai-settings

calendarCategories.js

Method	Path / registration template
GET	/api/calendar/categories
POST	/api/calendar/categories
PUT	/api/calendar/categories/:id
DELETE	/api/calendar/categories/:id

calendarColors.js

Method	Path / registration template
GET	/api/calendar/colors
PUT	/api/calendar/colors

calendarHistory.js

Method	Path / registration template
GET	/api/calendar/history
POST	/api/calendar/undo

calendarSeries.js

Method	Path / registration template
POST	/api/calendar-series
PUT	/api/calendar-series/:id
DELETE	/api/calendar-series/:id

coverLetterGeneration.js

Method	Path / registration template
POST	${base}/generate
POST	${base}/save-draft

f1Calendar.js

Method	Path / registration template
GET	/api/calendar/sports/f1
POST	/api/calendar/sports/f1/import

folderAttachments.js

Method	Path / registration template
PUT	${base}/:id/attachments
POST	${base}/:id/attachments

folderHierarchy.js

Method	Path / registration template
PUT	${base}/:id
DELETE	${base}/:id

generatedDocuments.js

Method	Path / registration template
POST	/api/documents/generate
POST	/api/documents/create-generated

globalSearch.js

Method	Path / registration template
POST	/api/search/reindex
GET	/api/search/tasks/:id
GET	/api/search

jobFileImport.js

Method	Path / registration template
POST	/api/jobs/extract-file

jobs.js

Method	Path / registration template
POST	/api/jobs/extract
GET	/api/jobs
GET	/api/jobs/:id
POST	/api/jobs
DELETE	/api/jobs/:id

mathRecognition.js

Method	Path / registration template
POST	/api/calculator/recognize

oneNoteImport.js

Method	Path / registration template
POST	/api/documents/import-onenote

pdfCleanup.js

Method	Path / registration template
POST	/api/pdfs/:id/cleanup

server.js

Method	Path / registration template
POST	/api/auth/register
POST	/api/auth/login
GET	/api/assistant/status
POST	/api/assistant/test
POST	/api/assistant/chat
GET	/api/account/preferences
PUT	/api/account/preferences
POST	/api/user/reset
GET	/api/calendar-entries
POST	/api/calendar-entries
PUT	/api/calendar-entries/:id
DELETE	/api/calendar-entries/:id
GET	/api/todos
POST	/api/todos
POST	/api/task-dependencies
POST	/api/todos/reorder
PUT	/api/todos/:id
POST	/api/todos/bulk-delete
DELETE	/api/todos/:id([a-fA-F0-9]{24})
DELETE	/api/todos/folder/:folderId
DELETE	/api/todos/clear-inbox
GET	/api/folders
PUT	/api/user/canvas
GET	/api/user/canvas
POST	/api/canvas/test
POST	/api/canvas/sync
POST	/api/documents/actions
GET	/api/pdfs
GET	/api/pdfs/:id
POST	/api/pdfs/:id/clean-copy
POST	/api/pdfs

Method	Path / registration template
PUT	/api/pdfs/:id/task-link
PUT	/api/pdfs/:id/content
GET	/api/pdfs/:id/original
PUT	/api/pdfs/:id/annotations
POST	/api/pdfs/:id/pages
DELETE	/api/pdfs/:id/pages/:pageNumber
PUT	/api/pdfs/:id
GET	/api/pdfs/:id/download
DELETE	/api/pdfs/:id
GET	/api/pdf-folders
GET	/api/health
POST	/api/calendar/import

study.js

Method	Path / registration template
GET	/api/study
POST	/api/study
PATCH	/api/study/:id
DELETE	/api/study/:id

taskDeletionRecovery.js

Method	Path / registration template
GET	/api/todos/deletion-history
POST	/api/todos/deletion-history/:id/undo

txstCalendar.js

Method	Path / registration template
GET	/api/calendar/txst-events
POST	/api/calendar/txst-events/refresh
PUT	/api/calendar/visibility

workspaceSettings.js

Method	Path / registration template
GET	/api/account/workspace-layout
PUT	/api/account/workspace-layout

Registration notes

Folder hierarchy and folder attachment templates register under /api/folders and /api/pdf-folders. Tailoring registers separate cover-letter and resume generation/save-draft paths. Route templates and helper-generated routes should be read with their registration calls in server.js; the literal inventory may omit routes constructed by helpers.

APPENDIX C

Rapid interview cheat sheet

If asked about...	Lead with...	Then disclose...
The product	Connected academic and career workspace	Feature breadth does not prove reliability
Architecture	React -> Express -> MongoDB	Not microservices or real-time collaboration
Ownership	Token plus owner-scoped queries	IDs alone are not authorization
Undo	Workflow-specific snapshots/history	No universal rollback; reset is permanent
PDF editing	Base file plus annotations and overlays	Not secure redaction or full Word layout
Performance	Canvas, bounded work, workers	No measured speedup without profiling
Palm rejection	Pointer type and gesture arbitration	No hardware/ML palm classifier
AI use	Language/recognition with restricted context	Review required; provider dependency
Math	AI transcribes; local parser computes	Wrong transcription can yield wrong result
Career tailoring	Source evidence plus template fidelity	No invented facts or ATS guarantee
Job imports	Structured data, layout/OCR, manual review	No universal accuracy or login bypass
Events	Public source fetch, cache and identity	Upstream changes and freshness limits
Scaling	Storage/queues/shared coordination next	Do not claim those are deployed
Testing	Describe a concrete validation plan	Do not invent passed checks or metrics

Before you walk in

Prepare a clean demo account, source documents, a reviewed output and a local-only fallback. Know one failure case, one technical tradeoff and one next improvement. Show evidence of your own contribution and be clear about adapted libraries and assistance.